Implement Least Privilege Access for a Cloud Account

Task 1

Create a new IAM user in your chosen cloud provider (AWS, Azure, or GCP) named 'foodieAppTester' and assign only the permissions needed to list storage buckets or containers, but not to create or delete them. Hint: Use built-in roles like 'Storage Viewer' or create a custom policy with only 'list' actions.

Steps followed:

1. Went to IAM → Users → "Create user" in the AWS Management Console and named the user "foodieAppTester", selecting programmatic access only since this is a test/tooling identity.

2. Instead of attaching a broad built-in role such as AmazonS3FullAccess, created a custom policy restricted to list-only S3 actions, so the user can see what buckets and objects exist without being able to create, upload, or delete anything.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListBucketsAndContentsOnly",
      "Effect": "Allow",
      "Action": [
        "s3:ListAllMyBuckets",
        "s3:ListBucket",
        "s3:GetBucketLocation"
      ],
      "Resource": "*"
    }
  ]
}
```

3. Named the policy "FoodieAppTesterListOnlyPolicy" and attached it directly to foodieAppTester, deliberately excluding s3:CreateBucket, s3:PutObject, s3:DeleteObject, and s3:DeleteBucket.

4. Verified the configuration by signing in as foodieAppTester and confirming that the buckets list loaded correctly, while an attempt to create a new bucket or delete an existing object failed with an "AccessDenied" error, confirming the permissions were scoped as intended.

Task 2

Audit the permissions of an existing IAM user or role in your cloud account and identify at least two permissions that are broader than necessary for a Zomato-style food delivery app backend (for example, full access to all services instead of just database and storage read/write).

Audit findings:

Principal Audited	Permission Found	Why It Is Broader Than Necessary
foodBackendRole	AdministratorAccess (full access to all AWS services)	The backend only needs to read/write order and menu data and access storage for images; it does not need to manage IAM, billing,

		networking, or any other unrelated AWS service.
foodBackendRole	dynamodb:* on Resource: * (all tables in the account)	The role only interacts with the ‘Orders’ and ‘MenuItems’ tables, but the wildcard resource grants it access to every DynamoDB table in the account, including tables belonging to unrelated services.

Both findings were captured using the IAM console's “Policy usage” and “Access Advisor” views, which showed that most of the granted service categories under AdministratorAccess had zero recorded usage by this role over the last 90 days — a strong signal that the access is unused and excessive.

Task 3

Modify the overly broad permissions you found in the previous task by editing the policy to restrict access only to the required services and actions (for example, allow only read/write to the food menu database and order storage, deny all other actions).

Before (over-permissive):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "*",
      "Resource": "*"
    }
  ]
}
```

After (least privilege applied):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "MenuAndOrdersTableAccess",
      "Effect": "Allow",
      "Action": [
        "dynamodb:GetItem",
        "dynamodb:PutItem",
        "dynamodb:UpdateItem",
        "dynamodb:Query"
      ],
      "Resource": [
        "arn:aws:dynamodb:*:*:table/Orders",
        "arn:aws:dynamodb:*:*:table/MenuItems"
      ]
    },
    {
      "Sid": "OrderStorageReadWrite",
      "Effect": "Allow",
      "Action": [
```

```
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::food-order-storage/*"
    },
    {
      "Sid": "ExplicitDenyEverythingElse",
      "Effect": "Deny",
      "NotAction": [
        "dynamodb:GetItem",
        "dynamodb:PutItem",
        "dynamodb:UpdateItem",
        "dynamodb:Query",
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "*"
    }
  ]
}
```

The revised policy replaces the wildcard Action/Resource with two tightly scoped statements — one for the Orders and MenuItems DynamoDB tables, and one for the food-order-storage S3 bucket — and adds an explicit Deny guardrail for every other action as a defense-in-depth measure, so that even a future accidental policy attachment cannot silently widen this role's effective access.

Task 4

Use ChatGPT or Copilot to generate a least-privilege IAM policy JSON for a user who only needs to upload profile pictures to a 'user-profile-pics' storage bucket. Review the generated policy and apply it to a test user. Hint: Ask the AI tool to explain each permission in the policy so you understand what it allows.

Steps followed:

1. Prompted the AI tool to generate an AWS IAM policy for a user who should only be able to upload new profile picture files to a bucket named ‘user-profile-pics’, with no ability to delete or overwrite other users' files, and asked it to explain each action included.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "UploadOwnProfilePictureOnly",
      "Effect": "Allow",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::user-profile-pics/${aws:username}/*"
    }
  ]
}
```

AI's explanation of each permission (reviewed for accuracy):

1. s3:PutObject — allows uploading (writing) a new object to the bucket; it does not allow reading, listing, or deleting existing objects, so the user cannot browse or remove anyone else's files.

2. Resource — scoped to “user-profile-pics/${aws:username}/*”, meaning the policy variable ${aws:username} is substituted with the requester's own IAM username at request time, so each user can only write inside their own folder/prefix and cannot upload into another user's path.

3. No s3:DeleteObject or s3:GetObject was included, since the task only requires uploading; read and delete permissions would be added separately (e.g., to a different role) only if genuinely needed.

4. Verified the AI's explanation against AWS's official S3 action reference to confirm it was accurate, then created a test user (“profilePicUploaderTest”), attached this policy, and confirmed the user could upload a file to their own prefix but received an “AccessDenied” error when attempting to upload into another username's folder.

Task 5

Prepare a 3-minute presentation or a 1-page document showing your IAM user/role configurations, the before-and-after policy changes, and suggest one improvement to further tighten security for a Flipkart-style cloud backend.

1-page summary document — key contents:

1. IAM configurations covered: foodieAppTester (list-only storage access), foodBackendRole (before: AdministratorAccess — after: scoped DynamoDB + S3 access with explicit deny guardrail), and profilePicUploaderTest (upload-only, restricted to own username prefix).

2. Before-and-after comparison: each configuration was presented side-by-side, showing the original wildcard Action/Resource values against the final scoped policy, along with the specific AccessDenied test results that proved the restriction was working as intended.

3. Suggested improvement for a Flipkart-style backend: introduce time-bound, temporary IAM roles (assumed via AWS STS) for any elevated administrative task — such as a data migration or emergency fix — instead of granting standing broad permissions to a role or user. Combined with mandatory MFA for role assumption and CloudTrail-based alerting on any AssumeRole call, this would ensure that even necessary high-privilege actions are time-limited, logged, and require an explicit, auditable justification rather than being silently available at all times.